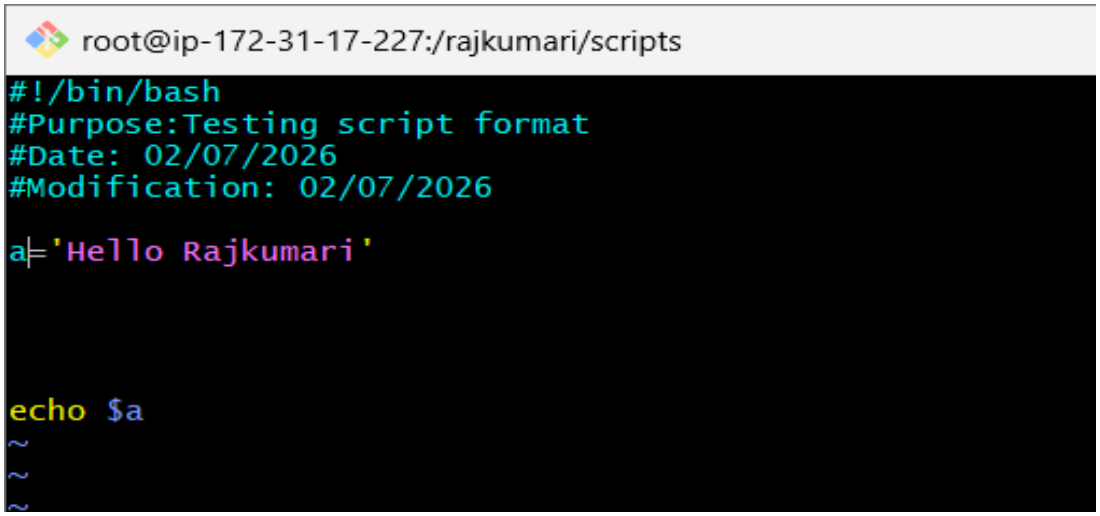
Bash Scripting: It is the process of writing a series of Linux commands in a text file called a shell script.
-> It is used to automate repetitive tasks, system administration, and application management.
-> Bash scripts are executed by the Bash (Bourne Again Shell) interpreter using the bash command.

<u>Script Format:</u> Steps which will help the other users to understand our script easily.

- Define shell -> **# !/bin/bash**
- Comments -> **# comments related to your script**
- Define variables -> **a='hello'**
- Commands -> **echo, cp, grep, etc..**
- Statements -> **if, while, for, etc..**



root@ip-172-31-17-227:/rajkumari/scripts

```
[root@ip-172-31-17-227 scripts]# ./helloworld.bash
Hello Rajkumari
[root@ip-172-31-17-227 scripts]# cat helloworld.bash
#!/bin/bash
#Purpose:Testing script format
#Date: 02/07/2026
#Modification: 02/07/2026

a='Hello Rajkumari'



echo $a
[root@ip-172-31-17-227 scripts]#
```

Testing Commands

root@ip-172-31-17-227:/rajkumari/scripts

```
#!/bin/bash
#owner= Rajkumari
#purpose: Testing commands
pwd
ls
whoami
date
cal
touch a b c
~
~
~
```

```
[root@ip-172-31-17-227 scripts]#  vi commands.bash
[root@ip-172-31-17-227 scripts]#  vi commands.bash
[root@ip-172-31-17-227 scripts]# chmod 755 commands.bash
[root@ip-172-31-17-227 scripts]# ./commands.bash
/rajkumari/scripts
abc.bash  commands.bash  helloworld.bash
root
Thu Jul  2 09:36:05 UTC 2026
      July 2026
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31

[root@ip-172-31-17-227 scripts]# cat commands.bash
#!/bin/bash
#owner= Rajkumari
#purpose: Testing commands
pwd
ls
whoami
date
cal
touch a b c
[root@ip-172-31-17-227 scripts]# ls
a  abc.bash  b  c  commands.bash  helloworld.bash
[root@ip-172-31-17-227 scripts]# whoami
root
[root@ip-172-31-17-227 scripts]# date
Thu Jul  2 09:38:05 UTC 2026
[root@ip-172-31-17-227 scripts]# cal
      July 2026
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31

[root@ip-172-31-17-227 scripts]#
```

The pwdi command is not a valid Linux command, so the Bash shell displays the error message CNF

```
root@ip-172-31-17-227:/rajkumari/scripts
[root@ip-172-31-17-227 scripts]# vi commands.bash
[root@ip-172-31-17-227 scripts]# pwdi
bash: pwdi: command not found
[root@ip-172-31-17-227 scripts]# ./commands.bash
./commands.bash: line 4: pwdi: command not found
a  abc.bash  b  c  commands.bash  helloworld.bash
root
Thu Jul  2 09:43:17 UTC 2026
      July 2026
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31

[root@ip-172-31-17-227 scripts]#
```

Basic Administration task

```
root@ip-172-31-17-227:/rajkumari/scripts
#!/bin/bash
#purpose: Admin task
#owner: Rajkumari
#date: 02-07-2026

top
df -h
free -m
uptime
iostat
~
~
~
```

The top command holds the terminal session while it is running. Once you press q to exit top, control returns to the Bash shell, and the remaining commands are executed.

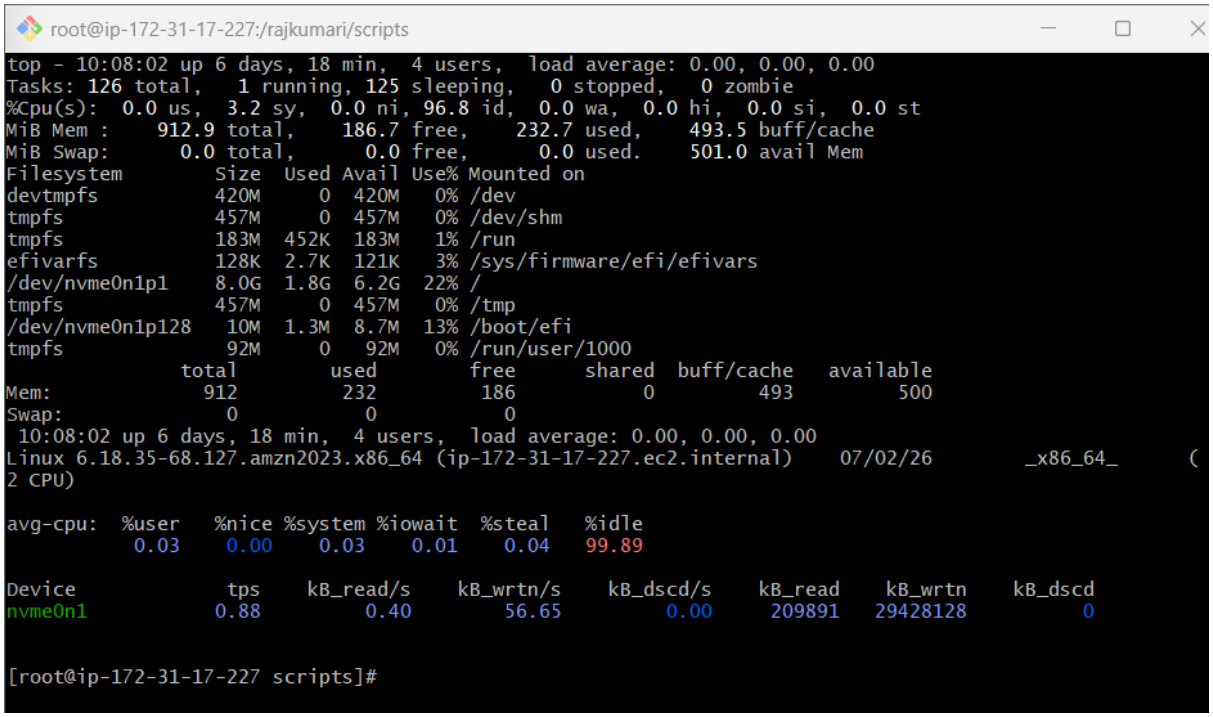
```
[root@ip-172-31-17-227 scripts]# vi admin.bash
[root@ip-172-31-17-227 scripts]# chmod 755 admin.bash
[root@ip-172-31-17-227 scripts]# ./admin.bash
top - 10:01:14 up 6 days, 11 min,  4 users,  load average: 0.00, 0.00, 0.00
Tasks: 127 total,   1 running, 126 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 99.7 id,  0.3 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    912.9 total,    186.5 free,    233.0 used,    493.3 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    500.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0  173220  17700  10976 S   0.0   1.9   0:26.30 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.10 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:01.52 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:05.40 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.20 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:02.06 migration/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     21 root      rt   0       0      0      0 S   0.0   0.0   0:02.08 migration/1
     22 root      20   0       0      0      0 S   0.0   0.0   0:01.61 ksoftirqd/1
     24 root       0 -20       0      0      0 I   0.0   0.0   0:00.95 kworker/1:0H-kblockd
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
     28 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-inet_frag_wq
     29 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
```

```
Filesystem        Size  Used Avail Use% Mounted on
devtmpfs          420M     0  420M   0% /dev
tmpfs             457M     0  457M   0% /dev/shm
tmpfs             183M  452K  183M   1% /run
efivarfs          128K  2.7K  121K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p1    8.0G  1.8G  6.2G  22% /
tmpfs             457M     0  457M   0% /tmp
/dev/nvme0n1p128   10M  1.3M  8.7M  13% /boot/efi
tmpfs              92M     0   92M   0% /run/user/1000
               total        used        free      shared  buff/cache   available
Mem:             912         233         186           0         493         500
Swap:              0           0           0
 10:03:11 up 6 days, 13 min,  4 users,  load average: 0.00, 0.00, 0.00
Linux 6.18.35-68.127.amzn2023.x86_64 (ip-172-31-17-227.ec2.internal)    07/02/26        _x86_64_      (
2 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
          0.03    0.00    0.03    0.01    0.04   99.89

Device             tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd
nvme0n1           0.88         0.40        56.66         0.00     209891   29418832          0


[root@ip-172-31-17-227 scripts]# ^C
[root@ip-172-31-17-227 scripts]#
```

The command top | head -5 shows only the first five lines of the top output and exits automatically. This allows the remaining commands in the script to execute without requiring manual input.

root@ip-172-31-17-227:/rajkumari/scripts

```
#!/bin/bash
#purpose: Admin task
#owner: Rajkumari
#date: 02-07-2026

top | head -5
df -h
free -m
uptime
iostat
~
~
```



Define variables: A variable is used to store data such as text, numbers, or command output for later use in the script.

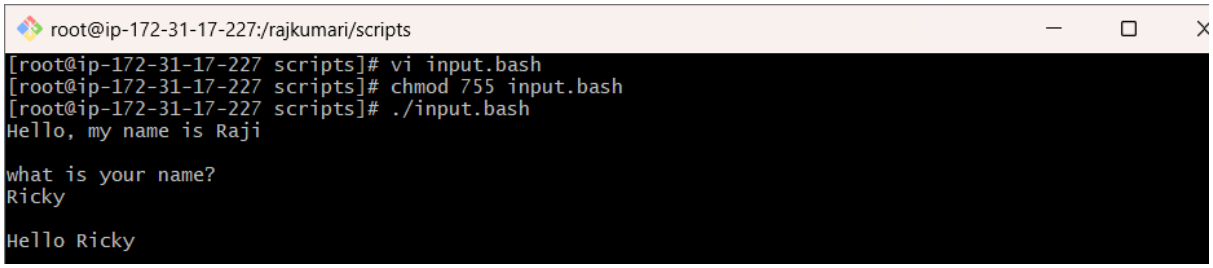
```
root@ip-172-31-17-227:/rajkumari/scripts
[root@ip-172-31-17-227 scripts]# ./var.bash
Run Variables tasks

/rajkumari/scripts
a  abc.bash  admin.bash  b  c  commands.bash  helloworld.bash  var.bash
root
Thu Jul  2 10:25:22 UTC 2026
                              2026

       January               February                March
Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa
             1  2  3    1  2  3  4  5  6  7    1  2  3  4  5  6  7
 4  5  6  7  8  9 10    8  9 10 11 12 13 14    8  9 10 11 12 13 14
11 12 13 14 15 16 17   15 16 17 18 19 20 21   15 16 17 18 19 20 21
18 19 20 21 22 23 24   22 23 24 25 26 27 28   22 23 24 25 26 27 28
25 26 27 28 29 30 31                          29 30 31

        April                  May                   June
Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa
          1  2  3  4                   1  2       1  2  3  4  5  6
 5  6  7  8  9 10 11    3  4  5  6  7  8  9    7  8  9 10 11 12 13
12 13 14 15 16 17 18   10 11 12 13 14 15 16   14 15 16 17 18 19 20
19 20 21 22 23 24 25   17 18 19 20 21 22 23   21 22 23 24 25 26 27
26 27 28 29 30         24 25 26 27 28 29 30   28 29 30
                       31
        July                 August                September
Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa
          1  2  3  4                      1          1  2  3  4  5
 5  6  7  8  9 10 11    2  3  4  5  6  7  8    6  7  8  9 10 11 12
12 13 14 15 16 17 18    9 10 11 12 13 14 15   13 14 15 16 17 18 19
19 20 21 22 23 24 25   16 17 18 19 20 21 22   20 21 22 23 24 25 26
26 27 28 29 30 31      23 24 25 26 27 28 29   27 28 29 30
                       30 31
       October              November              December
Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa
             1  2  3    1  2  3  4  5  6  7          1  2  3  4  5
 4  5  6  7  8  9 10    8  9 10 11 12 13 14    6  7  8  9 10 11 12
11 12 13 14 15 16 17   15 16 17 18 19 20 21   13 14 15 16 17 18 19
18 19 20 21 22 23 24   22 23 24 25 26 27 28   20 21 22 23 24 25 26
25 26 27 28 29 30 31   29 30                 27 28 29 30 31
```

Input script:

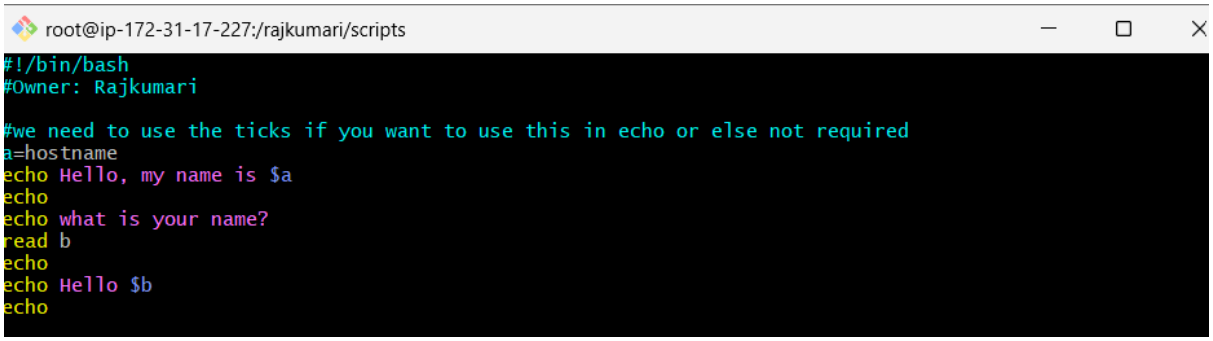
read --> Used to accept input from the user and store it in a variable.

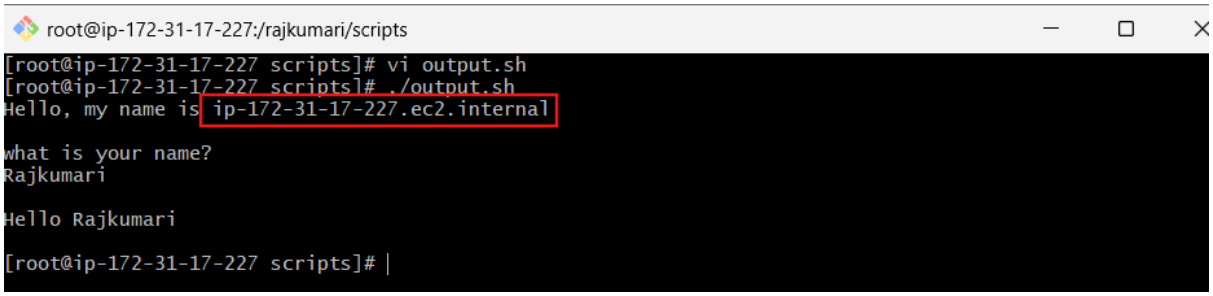
echo --> Used to display text, variable values, or command output on the terminal.

```
root@ip-172-31-17-227:/rajkumari/scripts
#!/bin/bash
#Owner: Rajkumari

echo Hello, my name is Raji
echo
echo what is your name?
read name
echo
echo Hello $name
echo

~
```



Output script:





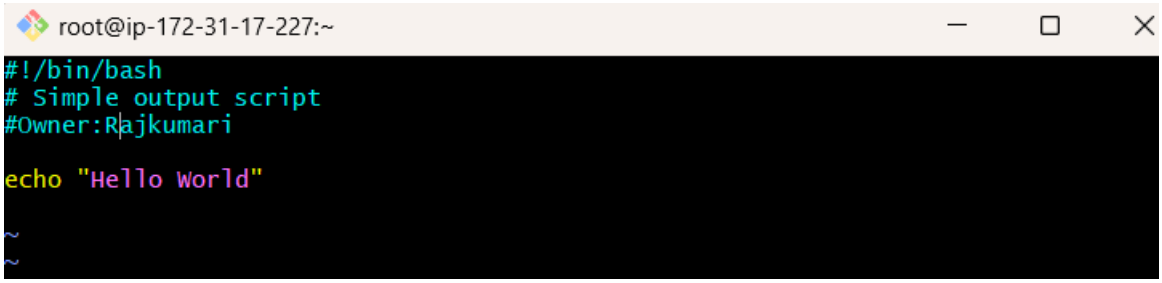
Basic Shell Scripts:

Sample 1 : Output to screen

Step:1 -> create a file # vi hello.bash

Follow the Script format steps

Save & quit



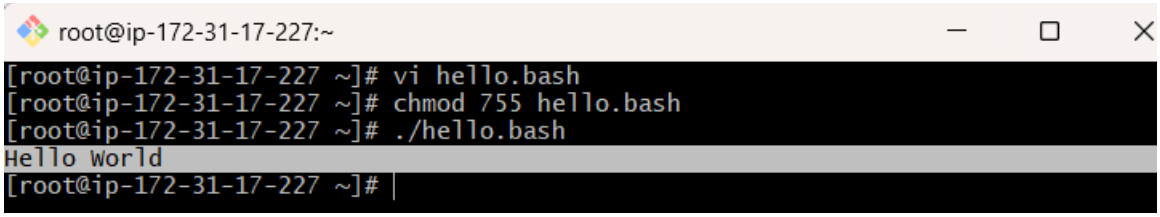
Step:2 Give the 'read' and 'execution' permissions by

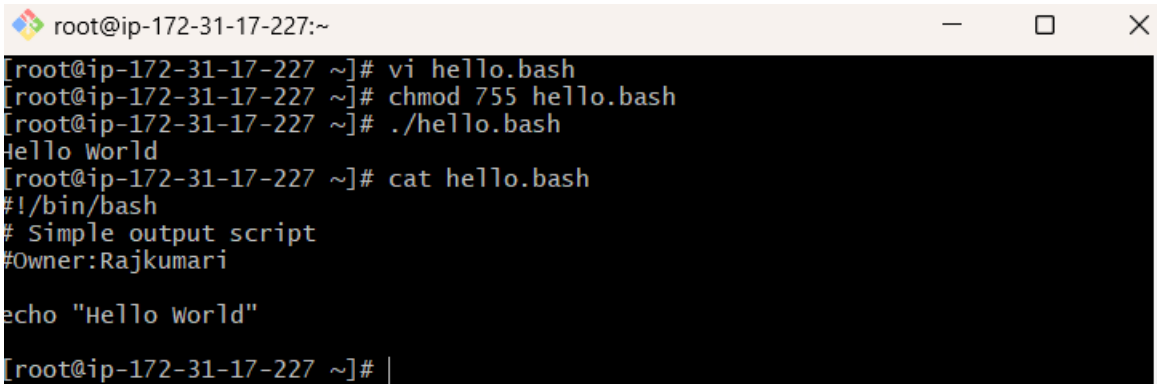
Using command

 # chmod 755 hello.bash

Step:3 Execute the Script

 # ./hello.bash





Sample:2 Executed bunch of commands in one script

--- > same steps as follow for all examples

```
[root@ip-172-31-17-227 ~]# vi cmd.bash
[root@ip-172-31-17-227 ~]# chmod 755 cmd.bash
[root@ip-172-31-17-227 ~]# ./cmd.bash
root

/root

ip-172-31-17-227.ec2.internal

total 8
-rw-r--r--. 1 root root  0 Jul  2 08:38 helloworld
-rwxr-xr-x. 1 root root 73 Jul  2 14:48 hello.bash
-rwxr-xr-x. 1 root root 91 Jul  2 14:58 cmd.bash

[root@ip-172-31-17-227 ~]# cat cmd.bash
#!/bin/bash
# Commands
#owner: Rajkumari

whoami
echo
pwd
echo
hostname
echo
ls -ltr
echo

[root@ip-172-31-17-227 ~]#
```

Sample: 3 on Variables

```
#!/bin/bash
# Example of defining variables

a=Rajkumari
b=Badugu
c='Devops'

echo "My first name is $a"
echo "My surname is $b"
echo "I am learning $c"

~
~
```

```
[root@ip-172-31-17-227 ~]# vi var.bash
[root@ip-172-31-17-227 ~]# chmod 755 var.bash
[root@ip-172-31-17-227 ~]# ./var.bash
My first name is Rajkumari
My surname is Badugu
I am learning 'Devops'
[root@ip-172-31-17-227 ~]#
```

Sample: 4 On Read Input

```
#!/bin/bash
# Read user input

echo "What is your favourite OS ?"
read a
echo

echo "What is your favourite Domain ?"
read b
echo
echo Hello $a $b
~
~
```

```
[root@ip-172-31-17-227 ~]# vi read.bash
[root@ip-172-31-17-227 ~]# chmod 755 read.bash
[root@ip-172-31-17-227 ~]# ./read.bash
What is your favourite OS ?
Linux

What is your favourite Domain ?
Devops

Hello Linux Devops
[root@ip-172-31-17-227 ~]#
```

Sample: 5 Scripts to run commands within

```
#!/bin/bash
# Script to run commands within

clear
echo "Hello `whoami`"
echo
echo "Today is `date`"
echo
echo "Number of user login: `who | wc -l `"
echo

~
```

o/p:

```
Hello root

Today is Thu Jul  2 16:09:39 UTC 2026

Number of user login: 2

[root@ip-172-31-17-227 ~]#
```

Sample 6: Read input and perform a task

```
#!/bin/bash
# This script will rename a file

echo Enter the file name to be renamed
read oldfilename

echo Enter the new file name
read newfilename

mv $oldfilename $newfilename
echo The file has been renamed as $newfilename
```

```
[root@ip-172-31-17-227 ~]# ls
cmd.bash  hello.bash  read.bash    run.bash
engineer  helloworld  rename.bash  var.bash
[root@ip-172-31-17-227 ~]# ./rename.bash
Enter the file name to be renamed
engineer
Enter the new file name
devopsengineer
The file has been renamed as devopsengineer
[root@ip-172-31-17-227 ~]#
```

‘If – then’ Scripts:

1. Check the variable

```
#!/bin/bash

count=100
if [ $count -eq 100 ]
then
  echo Count is 100
else
  echo Count is not 100
fi
```

```
[root@ip-172-31-17-227 ~]# vi if1.bash
[root@ip-172-31-17-227 ~]# chmod 755 if1.bash
[root@ip-172-31-17-227 ~]# ./if1.bash
Count is 100
[root@ip-172-31-17-227 ~]#
```

2. Check if a file error.txt exit

```
#!/bin/bash

clear
if [ -e /home/rajkumari/error.txt ]

        then
        echo "File exist"
        else
        echo "File does not exist"
fi
```

```
File does not exist
[root@ip-172-31-17-227 ~]#
```

```
[root@ip-172-31-17-227 ~]# su rajkumari
[rajkumari@ip-172-31-17-227 root]$ cd
[rajkumari@ip-172-31-17-227 ~]$ pwd
/home/rajkumari
[rajkumari@ip-172-31-17-227 ~]$ touch error.txt
[rajkumari@ip-172-31-17-227 ~]$ ls
Desktop  error.txt  project.txt
[rajkumari@ip-172-31-17-227 ~]$ ./if2.bash
```

```
File exist
[root@ip-172-31-17-227 ~]#
```

3. Check if a variable value is met

```
#!/bin/bash

a=`date | awk '{print $1}'`

if [ "$a" == Mon ]

        then
        echo Today is $a
        else
        echo Today is not Monday
fi
```

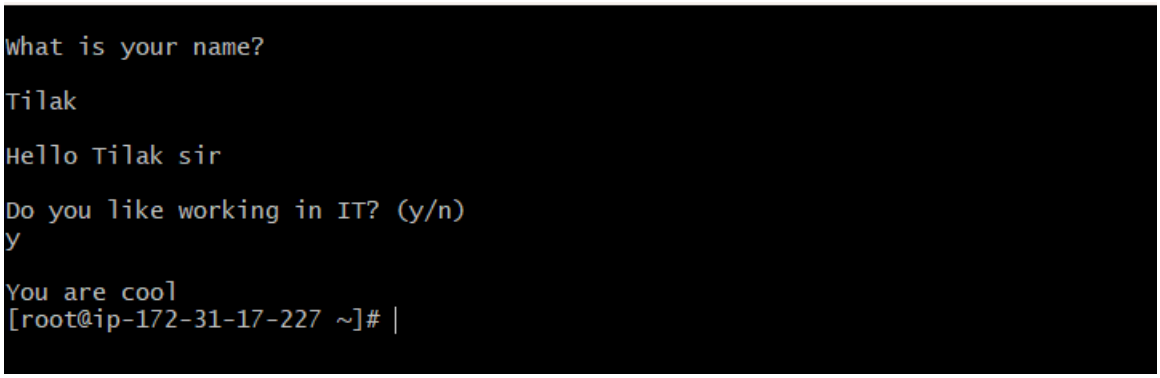
```
[root@ip-172-31-17-227 ~]# vi if3.bash
[root@ip-172-31-17-227 ~]# chmod 755 if3.bash
[root@ip-172-31-17-227 ~]# ./if3.bash
Today is not Monday
[root@ip-172-31-17-227 ~]# date
Thu Jul  2 16:59:21 UTC 2026
[root@ip-172-31-17-227 ~]#
```

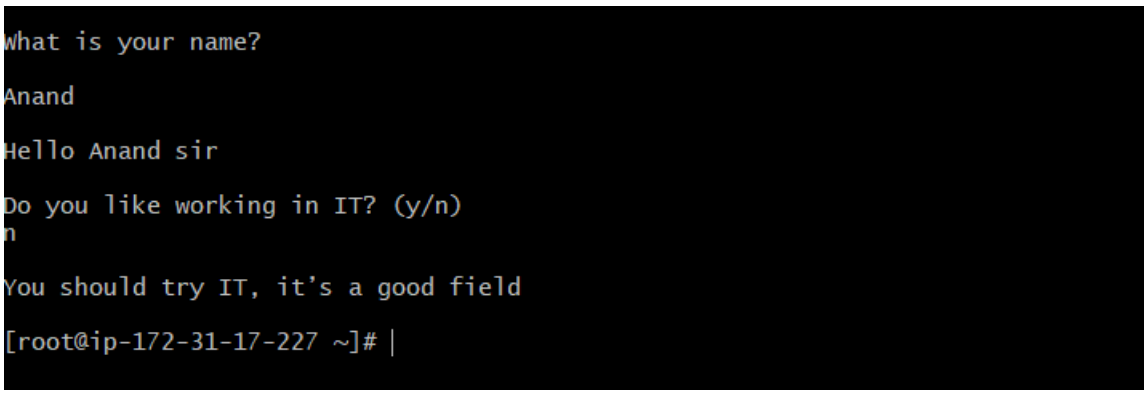
```
#!/bin/bash

a=`date | awk '{print $1}'`

if [ "$a" == Thu ]

        then
        echo Today is $a
        else
        echo Today is not Monday
fi
```

```
[root@ip-172-31-17-227 ~]# vi if3.bash
[root@ip-172-31-17-227 ~]# ./if3.bash
Today is Thu
[root@ip-172-31-17-227 ~]# date
Thu Jul  2 17:02:23 UTC 2026
[root@ip-172-31-17-227 ~]#
```

4. Check the response and then output

```
#!/bin/bash

clear
echo
echo "What is your name?"
echo
read a
echo

echo Hello $a sir
echo

echo "Do you like working in IT? (y/n)"
read Like
echo

if [ "$Like" == y ]
then
echo You are cool

elif [ "$Like" == n ]
then
echo You should try IT, it's a good field
echo
fi
```





More information about 'If' statements

If the output is either Monday or Tuesday

if ["$a" = Monday] || ["$a" = Tuesday]

Test if the error.txt file exist and its size is greater than zero

if test -s error.txt

if [$? -eq 0] -> If input is equal to zero (0**)**

if [-e /export/home/filename]-> If file is there

**if ["$a" != ""] -> ** If variable does not match

if [error_code != "0"] ->If file not equal to zero (0)

Comparisons:

-eq **equal to for numbers**

== **equal to for letters**

-ne **not equal to**

!== **not equal to for letters**

-lt **less than**

-le **less than or equal to**

-gt **greater than**

-ge **greater than or equal to**

File Operati:ons

-s **-→ file exists and is not empty**

-f **-→ file exists and is not a directory**

-d **-→ directory exists**

-x **-→ file is executable**

-w **-→ file is writable**

-r **--> file is readable**

For loop Scripts

Objective: The objective of a **For Loop** in a shell script is to automate repetitive tasks by executing the same set of commands multiple times for each item in a list, range, or collection.

It helps reduce manual effort, improves efficiency, and makes scripts easier to maintain.

Purpose: The purpose of a **For Loop** is to iterate through a predefined list of values, files, directories, or numbers and perform the required operation on each item automatically.

It is widely used in Linux administration and automation for tasks such as processing multiple files, creating users, taking backups, checking services, generating reports, and executing repetitive system administration tasks.

Use Cases

- Process multiple files or directories.
- Create or manage multiple users.
- Perform batch operations on servers.
- Execute commands repeatedly for a range of numbers.
- Automate backups and log management.
- Check the status of multiple services.

- Generate reports by looping through system information.
- Reduce manual work and improve script efficiency.

Simple for loop output: 1

```
#!/bin/bash

for i in 1 2 3 4 5
do
echo "Welcome $i times"
done
```

```
[root@ip-172-31-17-227 ~]# vi for1.bash
[root@ip-172-31-17-227 ~]# chmod 755 for1.bash
[root@ip-172-31-17-227 ~]# vi for1.bash
[root@ip-172-31-17-227 ~]# ./for1.bash
Welcome 1 times
Welcome 2 times
Welcome 3 times
Welcome 4 times
Welcome 5 times
[root@ip-172-31-17-227 ~]#
```

Simple for loop output: 2

```
#!/bin/bash

for i in eat run jump play
do
echo See Arunn $i
done
```

```
[root@ip-172-31-17-227 ~]# ./for2.bash
See Arunn eat
See Arunn run
See Arunn jump
See Arunn play
[root@ip-172-31-17-227 ~]#
```

for loop to create 5 files named 1-5

```
#!/bin/bash

for i in {1..5}
do
   touch $i
done
```

```
[root@ip-172-31-17-227 ~]# vi for3.bash
[root@ip-172-31-17-227 ~]# vi for3.bash
[root@ip-172-31-17-227 ~]# chmod 755 for3.bash
[root@ip-172-31-17-227 ~]# ./for3.bash
[root@ip-172-31-17-227 ~]# ls
1  4          devopsengineer  for3.bash   if1.bash  if4.bash     run.bash
2  5          for1.bash       hello.bash  if2.bash  read.bash    var.bash
3  cmd.bash   for2.bash       helloworld  if3.bash  rename.bash
[root@ip-172-31-17-227 ~]#
```

for loop to delete 5 files named 1-5

```
#!/bin/bash

for i in {1..5}
do
   rm $i
done
```

```
[root@ip-172-31-17-227 ~]# vi for4.bash
[root@ip-172-31-17-227 ~]# chmod 755 for4.bash
[root@ip-172-31-17-227 ~]# ./for4.bash
[root@ip-172-31-17-227 ~]# ls
cmd.bash        for2.bash  hello.bash  if2.bash  read.bash    var.bash
devopsengineer  for3.bash  helloworld  if3.bash  rename.bash
for1.bash       for4.bash  if1.bash    if4.bash  run.bash
[root@ip-172-31-17-227 ~]#
```

Specify days in for loop

```
#!/bin/bash

i=1
for day in Mon Tue Wed Thu Fri
do
 echo "Weekday $((i++)) : $day"
done
```

```
[root@ip-172-31-17-227 ~]# vi for5.bash
[root@ip-172-31-17-227 ~]# vi for5.bash
[root@ip-172-31-17-227 ~]# chmod 755 for5.bash
[root@ip-172-31-17-227 ~]# ./for5.bash
Weekday 1 : Mon
Weekday 2 : Tue
Weekday 3 : Wed
Weekday 4 : Thu
Weekday 5 : Fri
[root@ip-172-31-17-227 ~]#
```

List all users one by one from /etc/passwd file

```
#!/bin/bash

i=1
for username in `awk -F: '{print $1}' /etc/passwd`
do
 echo "Username $((i++)) : $username"
done
```

```
[root@ip-172-31-17-227 ~]# vi for6.bash
[root@ip-172-31-17-227 ~]# chmod 755 for6.bash
[root@ip-172-31-17-227 ~]# ./for6.bash
Username 1 : root
Username 2 : bin
Username 3 : daemon
Username 4 : adm
Username 5 : lp
Username 6 : sync
Username 7 : shutdown
Username 8 : halt
Username 9 : mail
Username 10 : operator
Username 11 : games
Username 12 : ftp
Username 13 : nobody
Username 14 : dbus
Username 15 : systemd-network
Username 16 : systemd-oom
Username 17 : systemd-resolve
Username 18 : rpc
Username 19 : libstoragemgmt
Username 20 : systemd-coredump
Username 21 : systemd-timesync
Username 22 : sshd
Username 23 : chrony
Username 24 : ec2-instance-connect
Username 25 : stapunpriv
Username 26 : rpcuser
Username 27 : tcpdump
Username 28 : ec2-user
Username 29 : rajkumari
Username 30 : Techie
```

do – while Script

Objective: To execute a block of code repeatedly until a specified condition becomes false, ensuring that the code is executed at least once.

Purpose:

The purpose of the **do-while** script is to perform repetitive tasks where the commands must execute at least one time before checking the condition.

It is commonly used for menu-driven programs, user input validation, and tasks that require at least one execution regardless of the condition.

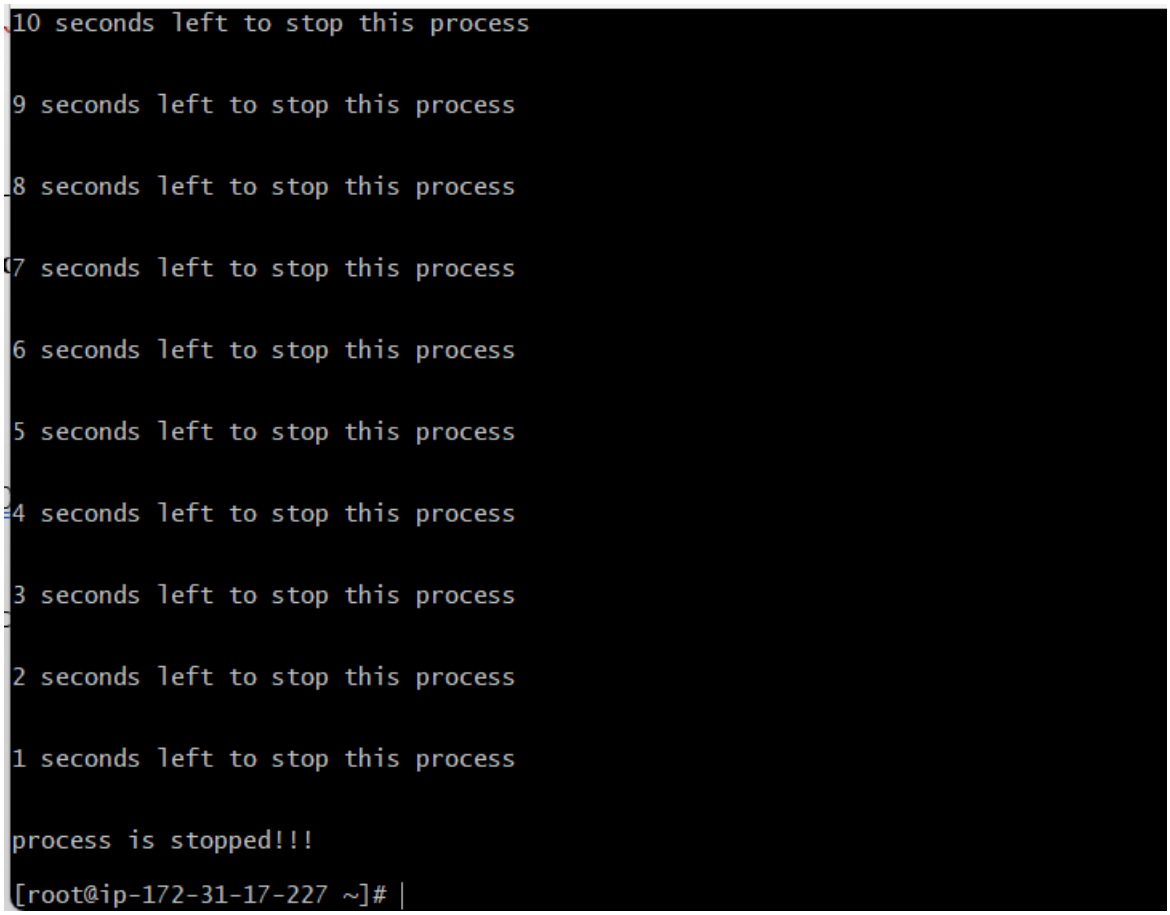
Script to run for a number of times

```
#!/bin/bash

c=1
while [ $c -le 5 ]
do
        echo "Welcone $c times"
        (( c++ ))
done

~
```

```
[root@ip-172-31-17-227 ~]# vi do1.bash
[root@ip-172-31-17-227 ~]# chmod 755 do1.bash
[root@ip-172-31-17-227 ~]# ./do1.bash
Welcone 1 times
Welcone 2 times
Welcone 3 times
Welcone 4 times
Welcone 5 times
[root@ip-172-31-17-227 ~]#
```

Script to run for a number of seconds

```
#!/bin/bash

count=0
num=10
while [ $count -lt 10 ]
do
        echo
        echo $num seconds left to stop this process $1
        echo
        sleep 1

num=`expr $num - 1`
count=`expr $count + 1`
done
echo
echo $1 process is stopped!!!
echo
```



case Script

Objective: To execute different blocks of code based on the value of a variable or user input.

Purpose: The purpose of the **case** script is to simplify decision-making by selecting and executing the appropriate block of code based on a matching pattern or user input.

It is commonly used for menu-driven programs, handling multiple choices, and improving the readability of shell scripts.

```
#!/bin/bash

echo
echo Please chose one of the options below
echo
echo 'a = Display Date and Time'
echo 'b = List file and directories'
echo 'c = List users logged in'
echo 'd = Check System uptime'
echo

        read choices

        case $choices in

a) date;;
b) ls;;
c) who;;
d) uptime;;
*) echo Invalid choice - Bye.
esac
```

```
[root@ip-172-31-17-227 ~]# vi case1.bash
[root@ip-172-31-17-227 ~]# ./case1.bash

Please chose one of the options below

a = Display Date and Time
b = List file and directories
c = List users logged in
d = Check System uptime

d
 18:32:30 up 6 days,  8:42,  2 users,  load average: 0.00, 0.00, 0.00
[root@ip-172-31-17-227 ~]#
```

This script will look at your current day and tell you the state of the backup

```
#!/bin/bash

NOW=$(date +"%a")
case $NOW in
        Mon)
                echo "Full backup";;
        Tue|Wed|Thu|Fri)
                echo "Partial backup";;
        Sat|Sun)
                echo "No backup";;
        *) ;;
esac
```

```
[root@ip-172-31-17-227 ~]# vi case2.bash
[root@ip-172-31-17-227 ~]# chmod 755 case2.bash
[root@ip-172-31-17-227 ~]# ./case2.bash
Partial backup
[root@ip-172-31-17-227 ~]#
```

Exit Status:

Objective: To determine whether a command or script executed successfully or failed by checking the return value (exit status).

Purpose: The purpose of the exit status is to provide feedback to the shell about the execution result of a command. It is widely used in Bash scripting for error handling, conditional statements (if, &&, ||), automation, and troubleshooting.

```
[root@ip-172-31-17-227 scripts]# pwd
/rajkumari/scripts
[root@ip-172-31-17-227 scripts]# echo $?
0
[root@ip-172-31-17-227 scripts]# pwdi
bash: pwdi: command not found
[root@ip-172-31-17-227 scripts]# echo $?
127
[root@ip-172-31-17-227 scripts]# mkpir
bash: mkpir: command not found
[root@ip-172-31-17-227 scripts]# echo $?
127
[root@ip-172-31-17-227 scripts]# ls
a         admin.bash  c             helloworld.bash  output.sh
abc.bash  b           commands.bash  input.bash       var.bash
[root@ip-172-31-17-227 scripts]# echo $?
0
[root@ip-172-31-17-227 scripts]# s
```

```
#!/bin/bash

echo hello
echo $?
~
```

```
[root@ip-172-31-17-227 scripts]# chmod 755 exit.bash
[root@ip-172-31-17-227 scripts]# ./exit.bash
hello
0
[root@ip-172-31-17-227 scripts]#
```

```
#Return exist status if file exist

#!/bin/bash

ls -l /home/rajkumari/check

if [ $? -eq 0 ]
then
echo File exist
else
echo File does not exist
fi
```

```
[root@ip-172-31-17-227 scripts]# vi exit2.bash
[root@ip-172-31-17-227 scripts]# chmod 755 exit2.bash
[root@ip-172-31-17-227 scripts]# ./exit2.bash
ls: cannot access '/home/rajkumari/check': No such file or directory
File does not exist
[root@ip-172-31-17-227 scripts]#
```

```
#Return exist status if file exist

#!/bin/bash

pwd

if [ $? -eq 0 ]
then
echo File exist
else
echo File does not exist
fi
```

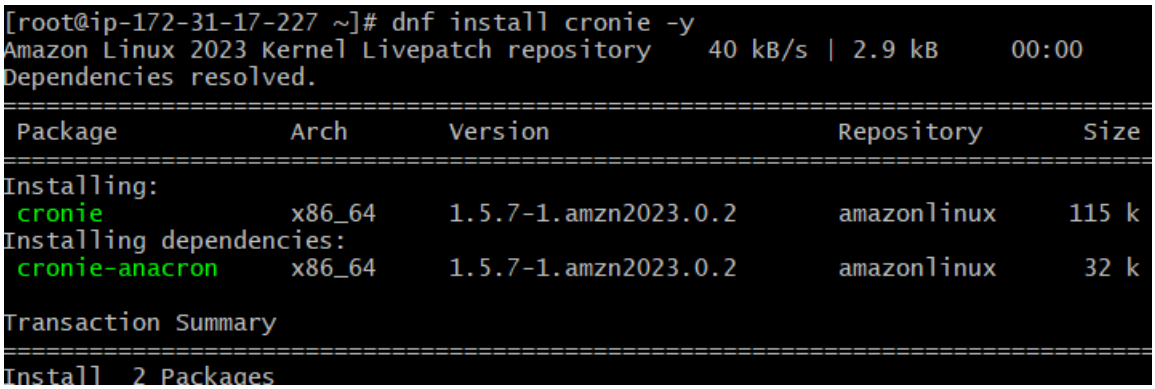
```
[root@ip-172-31-17-227 scripts]# ./exit2.bash
/rajkumari/scripts
File exist
[root@ip-172-31-17-227 scripts]#
```

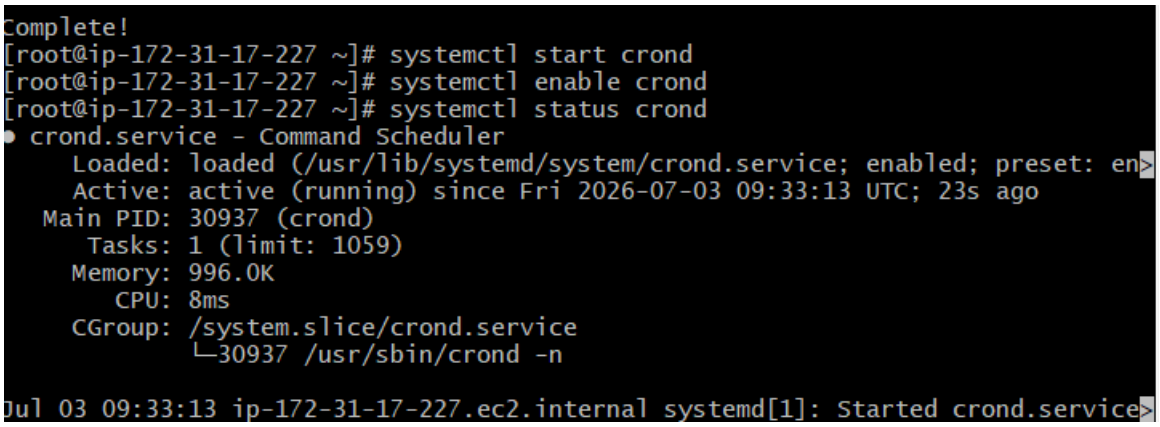
Crontab

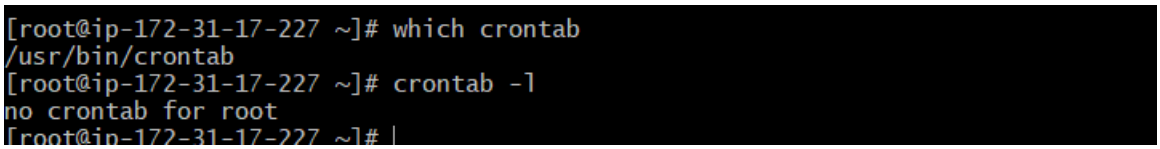
Objective: To automate the execution of commands or scripts at specific times, dates, or recurring intervals without requiring manual intervention.

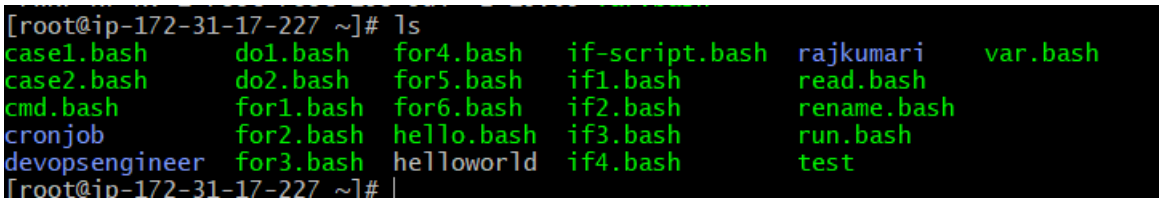
Purpose: The purpose of crontab is to save time and improve system administration by running scheduled jobs

automatically. It is commonly used for system maintenance, automation, backups, monitoring, and routine administrative tasks.










```
[root@ip-172-31-17-227 ~]# date
Sun Jul  5 03:53:05 UTC 2026
[root@ip-172-31-17-227 ~]# date
Sun Jul  5 03:54:42 UTC 2026
```